EE275 **Spring 2026**

San Jose State University
Department of Electrical Engineering

Mini-Project II (Due Mar. 31)

Mini-project II is to be carried out individually. In this mini-project, you are first to implement in Verilog/VHDL or in C/C++/Matlab/Python a subset of the MIPS instruction set architecture (ISA) that can execute the dot product benchmark program as below:

```
dot_product:
        addu $r1 $r0 $r0 ; result = 0
loop:
        beq $r7 $r0 done ; done looping?
        lw $r2 0($r3) ; load a elem
        lw $r4 0($r5) ; load b elem
        mul $r2 $r2 $r4 ; assume this is 1 instr.
        addu $r1 $r1 $r2 ; result += (*a) * (*b)
        addiu $r3 $r3 #4
        addiu $r5 $r5 #4
        addiu $r7 $r7 #-1
        j loop
done:
        jr $r31
```

r type

j type

i type

convert mips assembly into binary:

```
[1]  000000 00000 00000 00001 00000 100001
[2]  000100 00111 00000 0000 0000 0010 0000
[3]  100011 00011 00010 0000 0000 0000 0000
[4]  100011 00101 00100 0000 0000 0000 0000
[5]  000000 00010 00100 00010 00000 011000
[6]  000000 00001 00010 00001 00000 100001
[7]  001001 00011 00011 0000 0000 0000 0100
[8]  001001 00101 00101 0000 0000 0000 0100
[9]  001001 00111 00111 1111 1111 1111 1111
[10] 000010 00 0000 0000 0000 0000 0000 0001
[11] 000000 11111 000 0000 0000 0000 001000
```

After verifying your design, add the "forward chaining" technique as discussed in class. Then you should show by simulation how many cycles were reduced by adding the technique.

You may use ModelSim or any other equivalent CAD tool to design and show working simulations of your MIPS design for two vectors. Note that the instruction and data memory should be initialized with the program and data, respectively, in the test bench. The waveform from simulation should clearly show correct cycle-wise working of your design, as well as the correct dot product value at the end.

If using C/C++ or Marlab or Python, timing diagrams are not required, but the "status" of the CPU, i.e., value of all relevant registers, for each clock cycle (i.e., for each iteration) is to be shown.

For demonstration purpose, the first vector should be 9 single digits taken from your 9-digit student ID and the second vector the first vector added digit-wise by the last digit of your student ID. Note that 9 added by 3 is "wrapped around" to 2. For example, if your ID is 012345678, the first vector is (0,1,2,3,4,5,6,7,8) and the second vector (8,9,0,1,2,3,4,5,6), making dot product to 0×8+1×9+2×0+3×1+4×2+5×3+6×4+7×5+8×6. Working of your simulation for this demo case should be proven by the screen capture of the corresponding timing diagram or pipeline register status display at each clock cycle.

NOTE: To show the authenticity of your work, attach the last two digits of your student ID to every signal and variable name used in your Verilog/ VHDL/C/C++ code. For example, if your SID is 123456789, signal names should be x89, a89, etc. Each of the screen captures of your code and simulation results should bear these signal names.